



Bhartiya Vidya Bhavans
Sardar Patel Institute of
Technology
Munshi Nagar – Andheri West – 400058

Name: Shivaji Deshmukh
UID: 2024300044
Branch:CE

DIVISION: A
BATCH: C
DATE: 31/07/2026

Experiment No.:1

Aim :To implement Client Server communication via java RMI.

Theory :

To understand RMI, let's break down the four essential files from your snippets:

1. The Remote Interface (Calculator.java):

```
public interface Calculator extends Remote { ... }
```

Purpose: Acts as the shared contract or protocol between the Client and Server.

- **extends Remote:** Marking this interface with `java.rmi.Remote` signals to the Java runtime that its methods can be invoked from a remote JVM.
- **throws RemoteException:** Every remote method must declare `RemoteException` because network calls can fail due to connection issues, timeouts, or server drops.

2. The Remote Object Implementation (CalculatorImpl.java)

```
public class CalculatorImpl extends UnicastRemoteObject implements  
Calculator { ... }
```

Purpose: Contains the actual business logic executed on the server.

- **extends UnicastRemoteObject:** This superclass exports the object, enabling it to listen for incoming point-to-point RMI calls over socket connections.
- **Constructor:** Defines protected `CalculatorImpl()` throws `RemoteException` because calling `super()` exports the object, which can throw a network exception.

3. The RMI Registry & Server (Server . java)

```
Registry registry = LocateRegistry.createRegistry(1099);  
registry.rebind("ShivajiCalc", obj);
```

Purpose: Initializes the service and makes it discoverable.

- **LocateRegistry.createRegistry(1099):** Starts an RMI Registry directly inside the server process on port **1099** (the default RMI port).
- **registry.rebind("ShivajiCalc", obj):** Binds the instantiated implementation (obj) to a unique string key ("ShivajiCalc") in the registry. Think of the registry as a phonebook where "ShivajiCalc" is the name and obj is the phone number.

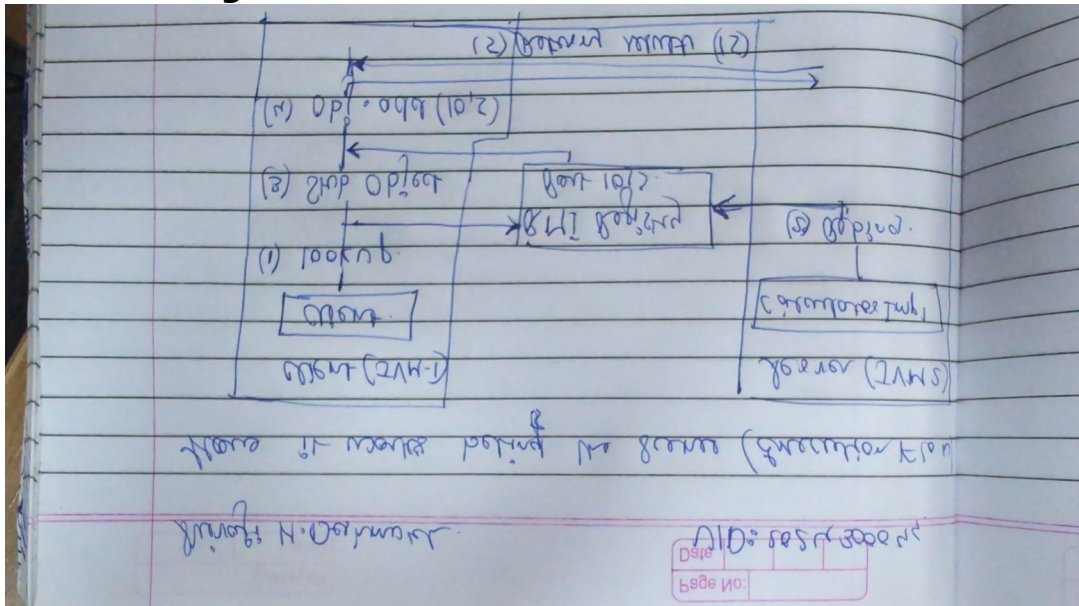
4. The Client (Client . java):

```
Registry registry = LocateRegistry.getRegistry("localhost", 1099);  
Calculator obj = (Calculator) registry.lookup("ShivajiCalc");
```

Purpose: Finds the remote service and requests computations.

- **LocateRegistry.getRegistry("localhost", 1099):** Connects to the RMI Registry running on the target machine (localhost) and port (1099).
- **registry.lookup("ShivajiCalc"):** Asks the registry for the object bound to "ShivajiCalc".
- **Remote Method Invocation:** Calling obj.add(10, 5) looks like a standard local method call, but behind the scenes, Java sends the parameters (10, 5) over the network to the server, computes $10 + 5 = 15$, and returns the result back to the client.

Problem-solving :



Code :

1) Calculator.java

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;

public interface Calculator extends Remote {
int add(int a,int b) throws RemoteException;
int subtract(int a,int b) throws RemoteException;
int multiply(int a,int b) throws RemoteException;
int divide(int a,int b) throws RemoteException;
}
```

2) CalculatorImpl.java

```
import java.rmi.*;
import java.rmi.server.*;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator{
protected CalculatorImpl() throws RemoteException{
super();
}
public int add(int a,int b) throws RemoteException{
return a + b;
}
public int subtract(int a,int b) throws RemoteException{
return a - b;
}
public int multiply(int a,int b) throws RemoteException{
return a * b;
}
public int divide(int a,int b) throws RemoteException{
return a / b;
}
}
```

3)Server.java

```
import java.rmi.registry.*;
public class Server {
public static void main(String[] args) {
try{
Calculator obj = new CalculatorImpl();
Registry registry = LocateRegistry.createRegistry(1099);
registry.rebind("ShivajiCalc", obj);
System.out.println("server is ready.....");
}catch(Exception ae){
System.out.println(ae);
}
}
}
```

4) Client.java

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {
public static void main(String[] args) {

try{
Registry registry = LocateRegistry.getRegistry("localhost",1099);
Calculator obj = (Calculator) registry.lookup("ShivajiCalc");
System.out.println("Addition = " + obj.add(10, 5));
System.out.println("Subtraction = " + obj.subtract(10, 5));
System.out.println("Multiplication = " + obj.multiply(10, 5));
System.out.println("Division = " + obj.divide(10, 5));

}catch(Exception e){
System.out.println(e);
}
}
}
}

```

Output screenshot :

The screenshot shows an IDE with several tabs: Calculator.java, CalculatorImpl.java, Server.java, Client.java, and CalculatorImpl.class. The active tab is Server.java, which contains the following code:

```

3 public class Server {
4     public static void main(String[] args) {
5         try{
6             Calculator obj = new CalculatorImpl();
7             Registry registry = LocateRegistry.createRegistry(1099);
8             registry.rebind("ShivajiCalc", obj);
9             System.out.println("server is ready.....");
10        }catch(Exception ae){
11            System.out.println(ae);
12        }
13    }
14 }
15

```

The terminal at the bottom shows the execution of the program. It displays the output of the Server.java file, which is "server is ready.....". It also shows the output of the Client.java file, which is:

```

Addition = 15
Subtraction = 5
Multiplication = 50
Division = 2

```

Conclusion: The interaction between your client and server through RMI demonstrates the core principles of distributed systems: network transparency, remote process communication, and interface-based design. By defining a shared Calculator interface, the server exposes its computational logic (CalculatorImpl) over the network without revealing implementation details. The RMI Registry acts as an intermediary directory, allowing the client to dynamically look up the remote service and execute methods like add or multiply as if they were running locally on its own machine. Overall, this experiment highlights how Java abstracts complex network socket programming into straightforward, object-oriented method calls across different JVMs.